

Task 9 – Theory & Written Answers

Part 01 – Enums, Classes, Generics, and OOP

Q1: Why is it recommended to explicitly assign values to enum members?

Explicit values should be assigned to enum members when:

- The enum is stored in a database or file and must remain stable across code versions.
- The enum is used in bitwise operations (flags), where powers of 2 are required.
- Interoperability with external systems requires fixed numeric values.
- Readability and intent clarity are important — readers should not need to count from the first value.

Without explicit values, adding a new member in the middle shifts all subsequent values, potentially breaking existing data or APIs.

Q2: What happens if you assign a value that exceeds the underlying type's range?

A compile-time error is raised. For example, if an enum has underlying type byte (range 0–255), assigning a value of 300 will cause the compiler to reject it. The value must fit within the numeric range supported by the underlying type. Choosing a narrower type saves memory but restricts the range of allowed values.

Q3: What is the purpose of the virtual keyword when used with properties?

The virtual keyword allows a property defined in a base class to be overridden in a derived class using the override keyword. This enables polymorphic behavior — different subclasses can provide different implementations of the same property. Without virtual, a derived class can only hide the property using new, which does not support polymorphism.

Q4: Why can't you override a sealed property or method?

A sealed member stops the inheritance chain at that level — it tells the compiler that no further override is allowed beyond this class. This is useful to:

- Protect critical logic from being accidentally changed in subclasses.
- Improve runtime performance by allowing the JIT compiler to devirtualize calls.
- Enforce design contracts where a specific behavior must be final.

Attempting to override a sealed member results in a compile-time error: 'cannot override inherited member because it is sealed'.

Q5: What is the key difference between static and object (instance) members?

Static members belong to the type itself and exist for the lifetime of the program. Instance members belong to a specific object and are created each time an object is instantiated.

- Static: called via `ClassName.Method()`; no object needed.
- Instance: called via `objectName.Method()`; requires an object.
- Static members share state across all usages; instance members are independent per object.
- Static classes cannot be instantiated and can only contain static members.

Q6: Can you overload all operators in C#? Explain why or why not.

No. C# allows overloading many operators but restricts others:

- Overloadable: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&`, `|`, `^`, `~`, `<<`, `>>`, `true`, `false`.
- Not overloadable: `=`, `&&`, `||`, `??`, `?:`, `checked`, `new`, `typeof`, `sizeof`, `is`, `as`.

The reason is that some operators (like `=`, `&&`, `||`) have deeply embedded language semantics. Allowing them to be overloaded could break fundamental language guarantees (e.g., short-circuit evaluation, assignment semantics) and lead to unpredictable or dangerous code.

Q7: When should you consider changing the underlying type of an enum?

- When memory is constrained (embedded systems, large arrays of enums): use `byte` or `short`.
- When the number of values exceeds the `int` range (very rare): use `long`.
- When interoperating with external systems that use a specific numeric type.
- When using the enum as a flags bitmask with many flags (may need `long` for >32 flags).

The default underlying type is `int` (4 bytes). Changing to `byte` (1 byte) saves 3 bytes per value — significant when storing millions of enum values in memory.

Q8: Why can't a static class have instance constructors?

A static class cannot be instantiated — no object of that type can ever be created. An instance constructor's sole purpose is to initialize a new object instance. Since no instances exist, an instance constructor is meaningless and the compiler disallows it. Static classes may only have a static constructor (used to initialize static fields), but never an instance constructor.

Q9: What are the advantages of `Enum.TryParse` over direct parsing with `int.Parse`?

- Safety: `TryParse` returns `false` for invalid inputs instead of throwing an exception.
- No try-catch needed: code remains cleaner and more readable.
- Works with both name and numeric string inputs ('A' or '5').
- Better performance: exception handling has overhead; `TryParse` avoids it entirely.

In user-facing applications where input cannot be guaranteed, `TryParse` is always the safer choice.

Q10: Difference between overriding Equals and == for object comparison (struct vs class)?

For classes (reference types):

- == by default checks reference equality (same memory address).
- Equals by default also checks reference equality but can be overridden for value equality.
- Overriding Equals allows comparing two different objects by their content.

For structs (value types):

- == is NOT provided by default for custom structs — it must be explicitly overloaded.
- Equals is provided via reflection-based comparison by default, but this is slow.
- Overriding both Equals and == for structs gives correct, fast value-based comparison.

Q11: Why is overriding ToString beneficial when working with custom classes?

- Produces meaningful output when an object is printed or converted to string.
- Without override, ToString returns the fully qualified class name — not useful.
- Aids debugging: Visual Studio's debugger uses ToString to display object values.
- Makes logging and display code cleaner — no need to manually format fields.
- Useful in string interpolation: \$"{employee}" calls ToString automatically.

Q12: Can generics be constrained to specific types in C#? Provide an example.

Yes. Generic constraints restrict which types can be used as type arguments. Common constraints:

- where T : IComparable — T must implement IComparable.
- where T : class — T must be a reference type.
- where T : struct — T must be a value type.
- where T : new() — T must have a parameterless constructor.
- where T : BaseClass — T must inherit from BaseClass.

Example: `static T Max(T a, T b) where T : IComparable { return a.CompareTo(b) >= 0 ? a : b; }`

This ensures CompareTo is available at compile time, providing type safety without casting.

Q13: What are the key differences between generic methods and generic classes?

- Scope: A generic method defines its own type parameter per call. A generic class defines the type parameter once at instantiation, shared by all methods inside.
- Usage: Generic methods are called with inferred or explicit type arguments. Generic classes require the type at object creation time.
- Flexibility: A generic method can live inside a non-generic class. A generic class makes all its members aware of the type parameter.

- Example: List is a generic class; Array.Sort() is a generic method.

Q14: Why might a generic swap method be preferable to custom swap methods per type?

- Eliminates code duplication — one method works for all types.
- Reduces maintenance burden — a bug fix in one place fixes all uses.
- Type safe — the compiler enforces correct usage at compile time.
- More readable and expressive — intent is clear regardless of the type swapped.

Q15: How can overriding Equals for Department improve search accuracy?

By default, Equals on a class compares references (memory addresses). Two Department objects with the same Name would not be considered equal. Overriding Equals to compare the Name field means that SearchArray will correctly find employees whose Department has the same name as the target, even if they are different object instances. Without this override, searches would always fail unless comparing the exact same object reference.

Q16: Why is == not implemented by default for structs?

Structs can have any combination of fields and types. The compiler cannot know in advance what 'equality' means for every possible struct — should it compare all fields? Only some? The designer's intent is unknown. Therefore, the compiler requires the developer to explicitly define == and != if value-based equality is desired. This is a deliberate design decision in C# to avoid silently incorrect comparisons.

Part 02 – Conceptual & Design Questions

1. LinkedIn Article Topic: Class Types in C#

Understanding Class Types in C#: A Practical Guide

C# offers several class types, each serving a distinct architectural purpose:

- **Concrete Classes:** Standard classes that can be fully instantiated. They implement all their methods and provide complete behavior.
- **Abstract Classes:** Cannot be instantiated directly. They serve as blueprints, providing shared base logic while forcing subclasses to implement abstract members.
- **Sealed Classes:** Cannot be inherited. Used when extension must be prevented, e.g., for security or design finality (example: `System.String`).
- **Static Classes:** Cannot be instantiated at all. Contain only static members. Used for utility/helper functions (e.g., `Math`, `Console`).
- **Partial Classes:** Split across multiple files. Useful in large codebases and auto-generated code (e.g., Windows Forms designer files).
- **Generic Classes:** Parameterized by type. Enable type-safe, reusable data structures (e.g., `List`, `Dictionary`).

Choosing the correct class type is a foundational OOP design decision that affects reusability, security, and maintainability.

2. What is the Generalization Concept Using Generics?

Generalization is the principle of writing code that works correctly for many types rather than being tied to one specific type. Generics in C# implement this principle at the language level.

Without generics, developers had to either:

- Write separate methods/classes for each type (duplication), or
- Use `object` as a universal parameter (losing type safety and causing boxing overhead).

With generics, a single definition such as `class Stack` or `T Max(T a, T b)` works for any type. The compiler enforces correct types at compile time, and no runtime casting is needed.

This embodies the Open/Closed Principle: the code is open for extension (new types) but closed for modification (no code changes needed). It leads to cleaner, safer, and more reusable designs — especially in collections, algorithms, and data structures.

3. What is Hierarchy Design in Real Business?

Hierarchy design in a real business context means modeling the relationships between entities using inheritance and composition to reflect the real-world organizational or domain structure.

In a company HR system, the hierarchy might be:

- **Person (base):** Name, Age, Address.

- Employee (extends Person): EmployeeId, Department, Salary.
- Manager (extends Employee): TeamSize, Budget.
- Executive (extends Manager): BoardSeat, StockOptions.

Benefits of hierarchy design:

- Code Reuse: Common fields/methods defined once in the base class.
- Polymorphism: A List can hold Employees, Managers, Executives.
- Maintainability: Changes to shared behavior happen in one place.
- Domain Clarity: Code mirrors business logic, making it easier to understand.

Good hierarchy design avoids deep chains (more than 3–4 levels) and prefers composition over inheritance when relationships are 'has-a' rather than 'is-a'.

Part 03 (Bonus) – Self Study & Event-Driven Programming

1. Self Study Report

Topics Studied: Enums, Classes, Generics, Operator Overloading, and OOP in C#

During this session, the following topics were explored in depth:

- **Enums:** Learned how to define enums with explicit values, change underlying types (byte, short), and use Enum.TryParse for safe parsing. Understood when explicit values are critical for data stability.
- **Classes and Properties:** Practiced adding properties to classes, using virtual to allow overriding, and sealed to prevent further overriding. Understood the design intent behind each keyword.
- **Static Members and Classes:** Explored the Utility pattern — a static class with no instances, used for pure helper functions like temperature conversion and perimeter calculation.
- **Operator Overloading:** Extended the ComplexNumber class with the * operator. Understood which operators can and cannot be overloaded and the reasoning behind restrictions.
- **Generics:** Implemented generic methods (Max, ReverseArray, FindMax) and a generic class (Stack, Helper2). Understood how generics provide type safety and eliminate code duplication.
- **Equals and ToString:** Overrode Equals in Employee and Department for content-based comparison. Saw the impact on SearchArray. Understood the difference between == and Equals for classes vs structs.
- **Structs vs Classes:** Created Circle as both a struct and a class. Compared behavior of == and Equals in both. Recognized why == must be explicitly implemented for structs.

Key Takeaway: C#'s type system is rich and intentional. Each keyword (virtual, sealed, static, generic constraint) exists to enforce correctness, performance, or design intent.

2. What is Event-Driven Programming?

Event-driven programming is a programming paradigm where the flow of execution is determined by events — user actions, messages from other programs, hardware signals, or system notifications — rather than a predetermined sequential flow.

Core concepts:

- **Event:** A signal that something happened (e.g., ButtonClicked, DataReceived, FileChanged).
- **Event Source (Publisher):** The object that detects or generates the event.
- **Event Handler (Subscriber):** A method registered to respond when the event fires.
- **Delegate:** In C#, a type-safe function pointer that points to the event handler method.
- **Event Keyword:** C# uses the event keyword to declare an event on a delegate type.

How it works in C#:

1. A class defines a delegate type and an event. 2. Other classes subscribe by attaching methods using +=. 3. When the event fires (Invoke), all subscribed handlers are called automatically.

Real-world examples:

- GUI Applications: Button clicks, mouse moves, key presses trigger UI updates.
- Web Servers: Incoming HTTP requests trigger route handlers.
- IoT Systems: Sensor readings trigger alert or logging handlers.
- Game Engines: Collision events trigger game logic responses.

Advantages:

- Decoupling: Publisher does not know who is listening — loose coupling.
- Scalability: Multiple handlers can respond to one event independently.
- Responsiveness: Programs respond to input as it arrives, not in fixed order.

In C#, events are built on delegates and follow the standard pattern: EventHandler for typed event data, ensuring a consistent and type-safe eventing model throughout the framework.